Phase 7 TSP Operator Discovery Protocol

This document defines the ambitious follow-up after the phase-6 mechanism study.

The goal is to stop asking only whether replay improves whole generated solvers and to ask a more interpretable question:

> *Can the system discover a compact, reusable TSP heuristic operator that survives transplant, ablation, and held-out validation?*

Central Question

The phase-7 win condition is not "beat Concorde" and not "invent a world-class solver."

The realistic win condition is:

> *The system discovers a compact, interpretable heuristic operator that gives a statistically reliable Pareto improvement over strong simple baselines on held-out TSP instance families.*

Why Modular Operators

Whole-solver evolution makes attribution hard:

- good final performance can hide known tricks, tuning, and accidental interactions
- replay may be helping search stability without producing a reusable idea
- code novelty is harder to interpret when every epoch can rewrite the full solver scaffold

Phase 7 therefore constrains the search object to one operator at a time.

Operator Basis

Phase 7 uses fixed operator interfaces implemented as bounded operator specifications returned by build_operator().

The interface is typed and range-bounded on purpose:

- candidate_ranker uses bounded float weights
- perturbation and restart_controller use bounded integer controls plus small enumerated mode sets
- candidate_pruner uses bounded integer controls rather than free-form continuous weights
- acceptance uses bounded float thresholds and temperatures
- scaffold_selector chooses only from the fixed scaffold pool

This is deliberate methodological discipline, not an implementation detail. It reduces output ambiguity, improves repeatability, and keeps the search object attributable instead of letting the LLM smuggle whole-solver behavior through an underspecified interface.

The current supported operator families are:

- candidate_ranker
- perturbation
- candidate_pruner
- acceptance

- restart_controller
- scaffold_selector for instance-adaptive solver selection across the fixed scaffold pool

This keeps the search object modular, attributable, and easy to transplant or ablate.

Research basis:

- Burke et al., Hyper-heuristics (https://econpapers.repec.org/bookchap/sprsprchp/978-3-319-07124-4_5f32.htm), 2018
- Smith-Miles and Lopes, Measuring instance difficulty for combinatorial optimization problems (https://www.cs.ubc.ca/labs/algorithms/EARG/stack/2012_ComputersAndOperationResearch_SmithMiles_MeasuringInstanceDifficulty.pdf), 2012
- Kerschke et al., A study on the effects of normalized TSP features for automated algorithm selection (https://www.sciencedirect.com/science/article/pii/S0304397522006089), 2022
- Selection-hyper-heuristic generality evidence: An investigation on the generality level of selection hyper-heuristics under different empirical conditions (https://www.sciencedirect.com/science/article/abs/pii/S1568494613000604), 2013
- Replication and repeated-run discipline remains the same as the earlier phases:
- Deep Reinforcement Learning That Matters (https://arxiv.org/abs/1709.06560)
- Rliable: Better Evaluation for Reinforcement Learning (https://arxiv.org/abs/2108.13264)

Scaffold Basis

Operators are validated against multiple simple solver scaffolds:

- nearest-neighbor + 2-opt
- cheapest insertion + 2-opt
- random restart + 2-opt
- sparse 3-opt-like baseline
- clustering + local search

The operator is the only thing that changes. The scaffolds remain fixed.

Condition Set

The official comparison set is:

1. baseline_heuristic_only
2. full_solver_evolution
3. modular_operator_evolution
4. modular_operator_random_replay
5. modular_operator_diversity_residual_replay
6. modular_operator_compression_pressure
7. modular_operator_pareto_selection

Interpretation target:

- whether modular operators are more reusable than whole-solver edits
- whether replay and compression still help when the evolved object is a single operator

- whether Pareto-aware selection uncovers operators that are not best on gap alone but are still scientifically useful

Validation Pipeline

Every final modular operator candidate must go through the following validation pipeline.

The automatic surviving_candidate flag in the suite output requires:

- positive transplant evidence across the configured scaffold set
- a clear family-specific gain without too many severe family regressions
- acceptable Pareto runtime inflation
- ablation evidence that disabling the operator hurts

A. Transplant Test

Insert the same operator into multiple independent scaffolds.

If the operator only helps in the host scaffold, it is likely overfit. If it helps across multiple scaffolds, it becomes more scientifically interesting.

B. Instance-Family Test

Evaluate separately on:

- held-out TSPLIB instances
- uniform Euclidean TSP
- clustered TSP
- grid-like TSP
- two-cluster bottleneck TSP
- elongated or corridor TSP
- adversarial nearest-neighbor-trap TSP

The operator does not need to help everywhere. It does need a clear domain of usefulness.

C. Pareto Test

Measure:

- optimality gap
- runtime
- number of distance evaluations
- code length
- operator complexity
- transfer to held-out families

The operator can be useful even if it is not the best pure-gap solution.

D. Ablation Test

Disable the operator and compare it against:

- the no-operator host baseline
- randomized or shuffled variants
- simplified variants
- the corresponding fixed host scaffold behavior

The aim is to show that the actual operator matters, not only the fact that the scaffold was rerun.

E. Independent Rediscovery Test

Across many seeds, record whether the same operator signature or operator family reappears independently.

This is not proof of novelty, but it is better evidence than a one-off code artifact.

Endpoints

Primary endpoint:

- final held-out TSPLIB mean optimality gap for the modular conditions

Secondary endpoints:

- family holdout gap
- transplant gain
- positive scaffold count
- Pareto runtime inflation
- distance-evaluation inflation
- accepted code novelty
- operator complexity
- adaptation efficiency
- rediscovery frequency

Interpretation Rules

- Do not claim operator discovery from one lucky seed.
- Do not claim a reusable operator unless it survives transplant or has a narrow but clear family-specific gain.
- Do not treat lower training gap alone as evidence of operator usefulness.
- Do not treat lexical novelty as discovery without the transplant and ablation evidence.
- If no operators survive, the pipeline acts as a falsification filter rather than a discovery amplifier.

Replication

Official main-study target:

- 20 paired seed offsets with the fixed list 0, 1000, 2000, ..., 19000

Keep the benchmark manifest, scaffold set, and validation pipeline fixed across compared conditions.

Deliverable

The required deliverable is an operator-discovery report for each surviving candidate:

- operator name
- core idea
- problem class where it helps
- pseudocode
- complexity
- why it should help
- failure cases
- ablation results
- transplant results
- held-out benchmark results
- novelty classification

Operational Entry Points

- Prepare the benchmark manifest with prepare_tsp_benchmarks.py
- Run the suite with run_tsp_operator_suite.py and configs/tsp_phase7_suite/01_operator_discovery.json
- Use configs/tsp_phase7_suite/RUNBOOK.md for concrete commands
- Aggregate repeated runs with aggregate_tsp_operator_runs.py
- Official results note: docs/PHASE_7_TSP_OPERATOR_DISCOVERY_RESULTS_2026-05-20.md

Relationship To Phase 6

Phase 6 is the mechanism study that explains the replay result. Phase 7 is the operator-discovery track that asks whether the system can produce a reusable operator primitive rather than only a tuned whole-solver scaffold.